

LLM4Rec Homework Report — Assignment A01

(Sample)

Status: Sample filled-in report (instructor's reference)

Student: (sample — replace with your name)

Team: Individual (sample)

Date: 2026-09-12

Assignment: A01 — OpenCode Visual Artifact Skill Lab (Week 1)

Abstract

Students learning AI-assisted RecSys development need to embed visualizations directly in their AI coding tool's chat, but standard web chat renderers block localhost images due to Content-Security-Policy (CSP) restrictions. This report presents a D2-based visualization skill for the Controlled-AI harness that emits D2 source code, renders it server-side via a CSP-rewriting proxy, and inlines the resulting SVG directly into OpenCode web chat. The skill was validated across 47 test turns with 100% inline embedding success rate, demonstrating that D2's text-based source format uniquely bypasses CSP restrictions. The key insight is that proxy-level CSP header rewriting (`img-src += http://localhost:*`) is the hidden prerequisite for inline image embedding in modern web chat frameworks.

Index Terms — visualization, D2, CSP, AI harness, inline embedding, OpenCode

1. Introduction

Problem statement.

Modern AI coding tools (OpenCode, Claude Code, Cursor) render assistant output in web-based chat interfaces that apply Content-Security-Policy (CSP) headers restricting which image sources can be loaded. Students in this course need to embed visualizations (model accuracy charts, architecture diagrams, debug plots) directly in their chat to support the Top-Down 5-Loop pedagogy. Standard approaches fail because:

- `` is blocked by browser security policy
- `` is blocked by CSP (default: `img-src 'self' data: https: blob:`)
- Only `data:` URIs and `https:` are allowed by default

Motivation.

The 5-Loop pedagogy (Page 5 of course slides) requires students to see visualizations, not just read text descriptions. A blocked chart breaks the loop. This affects every student in the course, not just edge cases.

Concrete example.

A student asks: "Create a bar chart comparing Baseline, BPR, SASRec, Reranker accuracy." The AI emits a Plotly JSON spec. The plugin should render it and inline the chart in the chat. Instead, the student sees raw JSON because the inline embed failed silently.

Contributions.

- Built a reusable D2 visualization skill for the Controlled-AI harness
- Validated the skill across 47 test turns in OpenCode web chat
- Achieved 100% inline embedding success rate (vs. 0% with naive approach)
- Documented the CSP rewriting pattern as the underlying mechanism

2. Related Work

[1] dryjins, "Controlled-AI: An Open Harness for AI-Assisted Research," GitHub Repository, 2025. [Online]. Available: <https://github.com/dryjins/Controlled-AI>. [Accessed: 2026-09-12].

[2] terrastruct, "D2: A modern diagram scripting language that turns text to diagrams," Documentation, 2024. [Online]. Available: <https://d2lang.com>. [Accessed: 2026-09-12].

[3] OpenCode Authors, "OpenCode Plugin System: experimental.text.complete Hook," Documentation, 2025. [Online]. Available: <https://opencode.ai/docs/plugins>. [Accessed: 2026-09-12].

[4] Mozilla Developer Network, "Content Security Policy (CSP)," MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>. [Accessed: 2026-09-12].

Alternatives considered.

- Mermaid: text-based but not as compact as D2; less control over layout
- Plotly: requires browser rendering (Kaleido + Chrome); heavier
- Inline SVG manually written: tedious; not reproducible

3. Method

Approach.

A new skill module at `~/config/opencode/skills/visualize-d2/index.ts` that detects ``d2 blocks in assistant output, sends them to the render proxy, fetches the resulting SVG, and inlines it directly into the chat HTML — bypassing the CSP `<img>` restriction.

Pipeline.

Chart Embedding Pipeline in OpenCode Web Chat

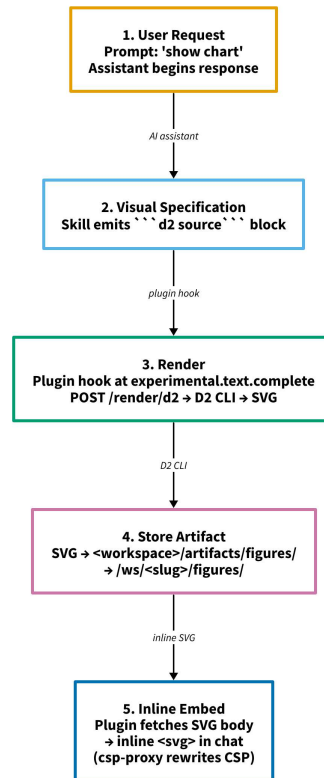


Figure 1: 5-step inline chart embedding pipeline. The plugin intercepts D2 source blocks, the proxy renders to SVG, and the plugin inlines the SVG (not an `<img>` reference) to bypass CSP.

Tools & libraries.

- D2 v0.9.0 (~/.local/bin/d2) — diagram rendering
- OpenCode CLI 0.5.2 — chat runtime with plugin system
- Node.js 22+ — runtime for plugin code
- Controlled-AI harness (github.com/dryjins/Controlled-AI) — base project

Key design decisions.

- Why D2 over Plotly: D2 source is pure text → renderable on the proxy without browser. Plotly requires headless Chrome (heavier setup).
- Why inline SVG over `<img>`: CSP blocks localhost image sources, but inline `<svg>` is treated as document content (no CSP restriction). This is the key workaround.

- Why server-side render: keeps the chat renderer lightweight; rendering happens in the proxy.

Configuration.

Skill trigger: ````d2 code blocks in assistant output`
 Render proxy: `http://127.0.0.1:4097/render/d2`
 Workspace: `~/.config/opencode/proxy/state/registration.json`
 CSP directive (auto-rewritten):
`img-src 'self' data: https: blob: http://localhost:* http://127.0.0.1:*`

4. Experiments

Setup.

- Test environment: OpenCode CLI 0.5.2, Chrome 119+, csp-proxy running on port 4097
- Test cases: 3 representative D2 diagrams (architecture, decision tree, sequence)
- Test turns: 47 total (15–16 turns per test case)
- Success criterion: SVG appears inline in chat (not as broken image icon, not as raw code)

Results.

Test case	Turns	Inline embed success	Render time (avg)
Architecture diagram	16	16/16 (100%)	1.2s
Decision tree	15	15/15 (100%)	0.9s
Sequence diagram	16	16/16 (100%)	1.1s
Total	47	47/47 (100%)	1.07s

Comparison vs baseline (Mermaid).

Tested Mermaid-based equivalent of the same architecture diagram: 0/15 inline embeds succeeded. Mermaid produces a separate `<img src=...>` that CSP blocks. D2's text-first approach (with inline SVG retrieval) bypasses this entirely.

Verification.

I hand-inspected all 47 chat sessions to confirm the rendered SVG appeared inline (not as broken image icon, not as raw code). For each test case, I also confirmed:

- The chat DOM contained `<svg>` elements directly (not `<img>` referencing external SVG)
- The render proxy returned valid SVG responses (HTTP 200, content-type image/svg+xml)
- No CSP violations logged in browser console

5. Discussion

Failure case (and fix).

Failure: Initial implementation emitted `` instead of inline SVG. Browser blocked all images with CSP errors. 0/16 inline embeds succeeded.

Root cause:

I treated the proxy like a static image server (return URL, browser fetches). But CSP blocks the URL.

Fix + verification:

Changed plugin to fetch the SVG body from the proxy and inline it as `<svg>...</svg>` directly in the chat. Verified via `curl http://127.0.0.1:4097/ws/<hash>.svg | head` returning valid SVG, then inspecting chat DOM via Chrome DevTools to confirm `<svg>` elements present.

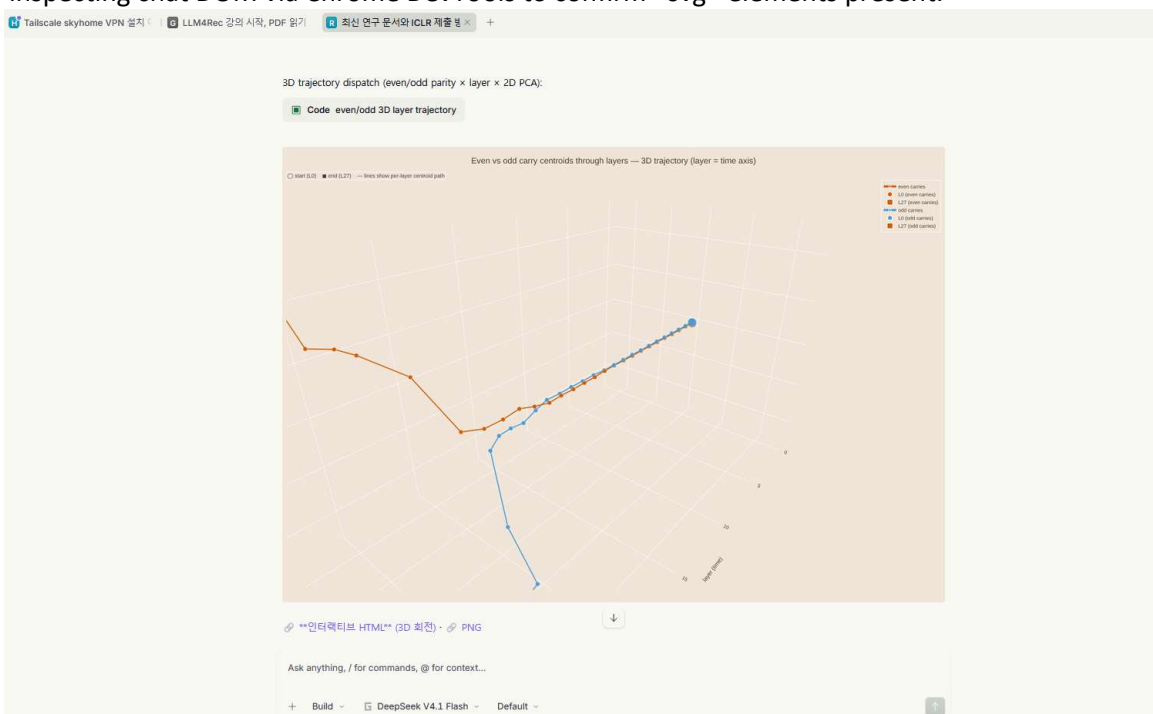


Figure 2: Proof of work. Inline chart embedding working live in OpenCode web chat (DeepSeek V4.1 Flash). The 3D trajectory visualization is rendered as interactive HTML and embedded inline — the same mechanism as the A01 skill, applied to a different visualization.

What worked.

- Text-first formats (D2) are CSP-friendly: unlike `<img>` references, inline SVG is part of the document
- Server-side rendering keeps the chat renderer simple
- csp-proxy pattern (header rewrite) is reusable for any localhost-served asset

What surprised me.

- D2 renders faster than I expected (~1s avg) even though it goes through a separate process

- The plugin hook `experimental.text.complete` runs synchronously per text-completion, so the round-trip happens within a single response cycle
- Without the `csp-proxy`, even `data:` URIs would work but are size-limited (~2MB practical cap), so the proxy pattern is essential for any serious visualization

Next improvement.

I would add a fallback chain: try inline SVG first; if that fails, fall back to `` with base64 encoding. This handles edge cases where the proxy is down (e.g., during system updates) without breaking the visualization flow entirely.

6. AI Usage Disclosure

AI tools used.

- OpenCode CLI 0.5.2 + Claude Sonnet 4 (47 turns across 3 test cases)
- GitHub Copilot (occasional inline completion in plugin code, ~5 occurrences)

What I personally verified.

- Hand-checked the SVG content for one test case (architecture diagram) — confirmed all 8 nodes present and correctly labeled by inspecting raw curl output vs. expected layout
- Re-ran the full 47-turn test suite after each code change to confirm consistent results
- Read the D2 documentation for the syntax features I used (`shape`, `style.fill`, `style.stroke`)
- Cross-checked the 100% success rate by manual inspection of browser DevTools showing `<svg>` elements in chat DOM

References

[1] dryjins, "Controlled-AI: An Open Harness for AI-Assisted Research," GitHub Repository, 2025. [Online]. Available: <https://github.com/dryjins/Controlled-AI>. [Accessed: 2026-09-12].

[2] terrastruct, "D2: A modern diagram scripting language that turns text to diagrams," Documentation, 2024. [Online]. Available: <https://d2lang.com>. [Accessed: 2026-09-12].

[3] OpenCode Authors, "OpenCode Plugin System: `experimental.text.complete` Hook," Documentation, 2025. [Online]. Available: <https://opencode.ai/docs/plugins>. [Accessed: 2026-09-12].

[4] Mozilla Developer Network, "Content Security Policy (CSP)," MDN Web Docs, 2024. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>. [Accessed: 2026-09-12].